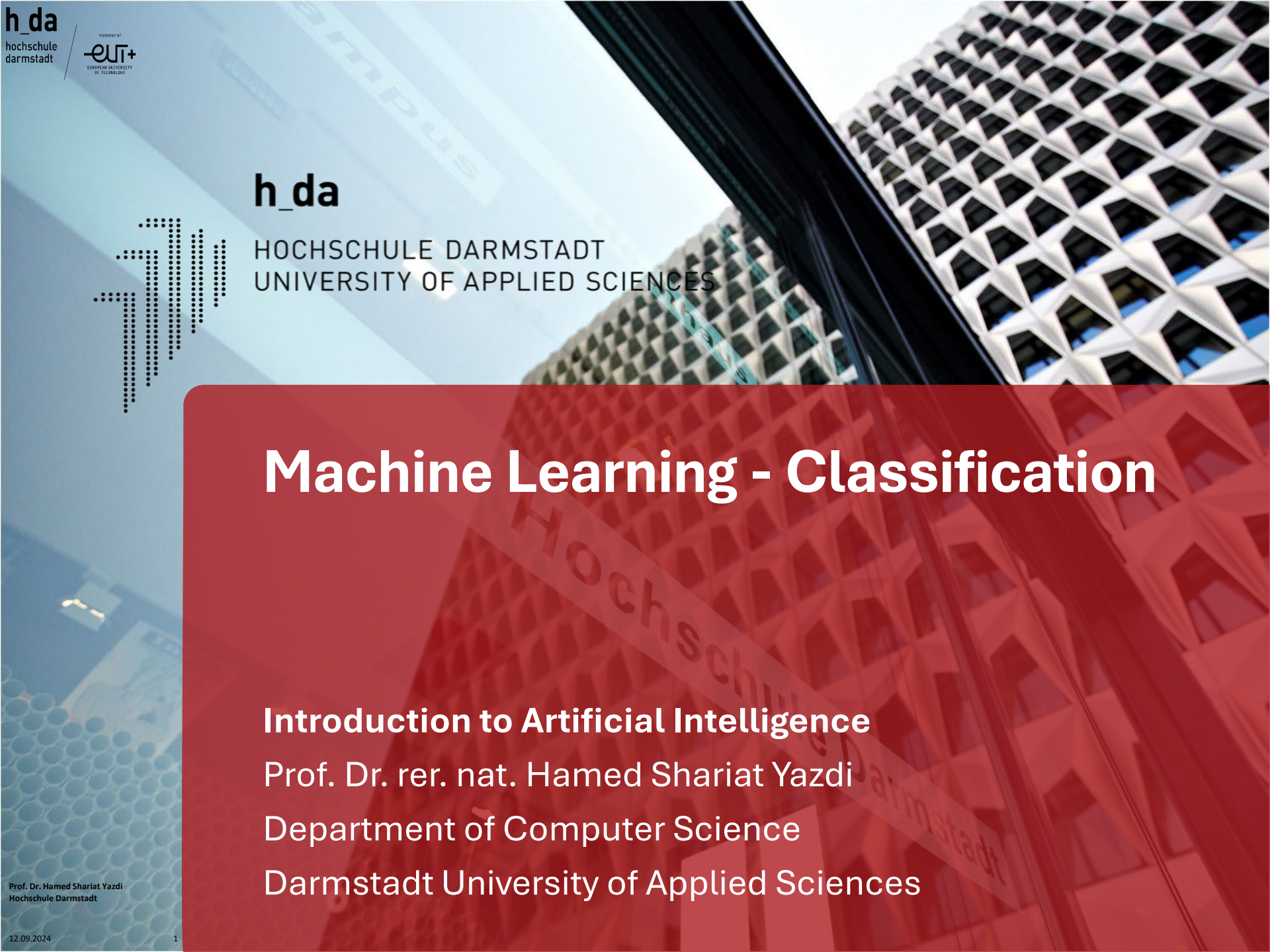


Machine Learning - Classification

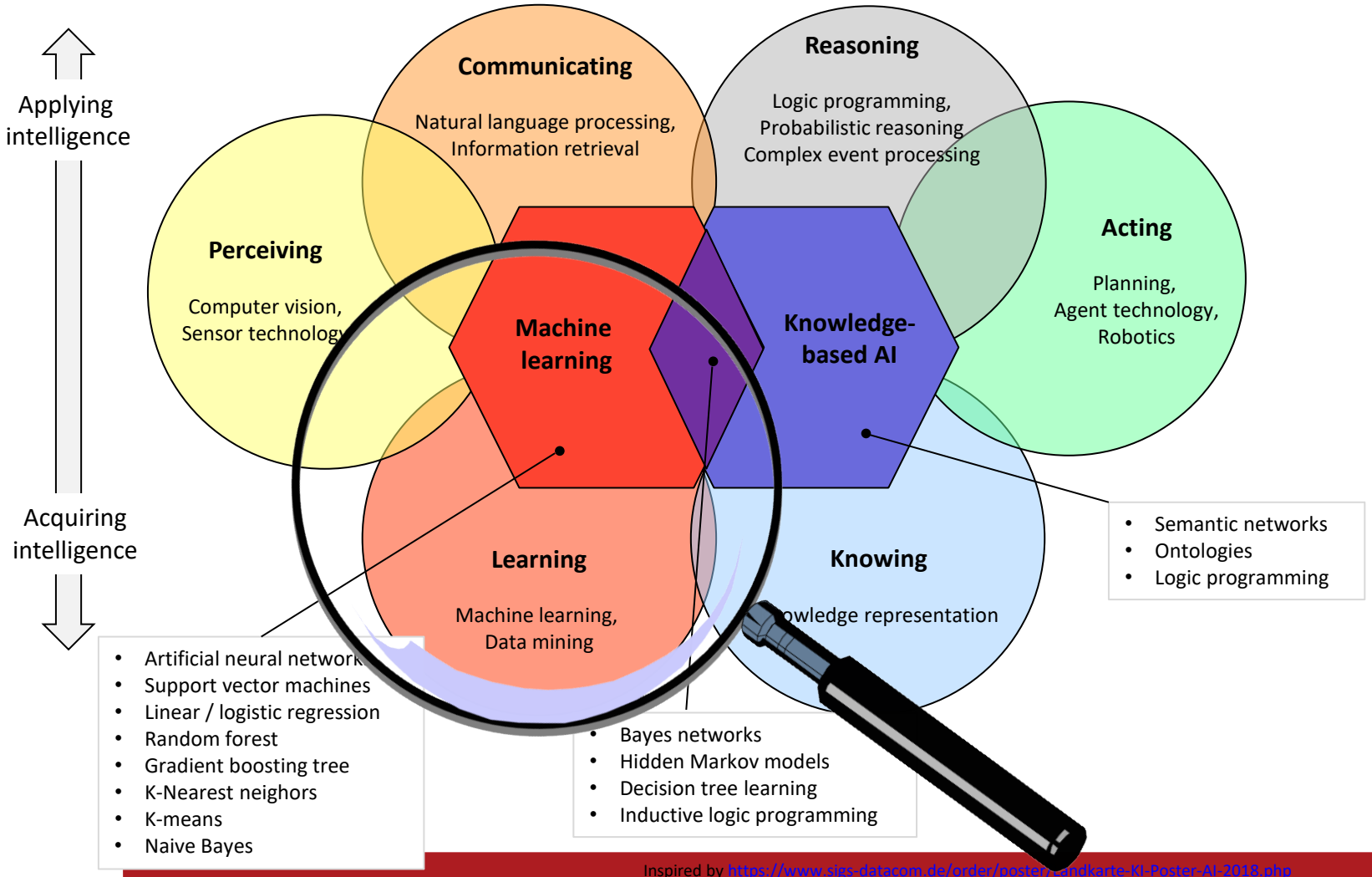
Introduction to Artificial Intelligence

Prof. Dr. rer. nat. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt University of Applied Sciences

Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Overview of dataset
- Loading data
- Exploring dataset
- Preprocessing data
- ML training
- Predicting
- Quick check

AI landscape



Inspired by https://www.sigs-datacom.de/order/poster/Landkarte-KI-Poster-AI-2018.php

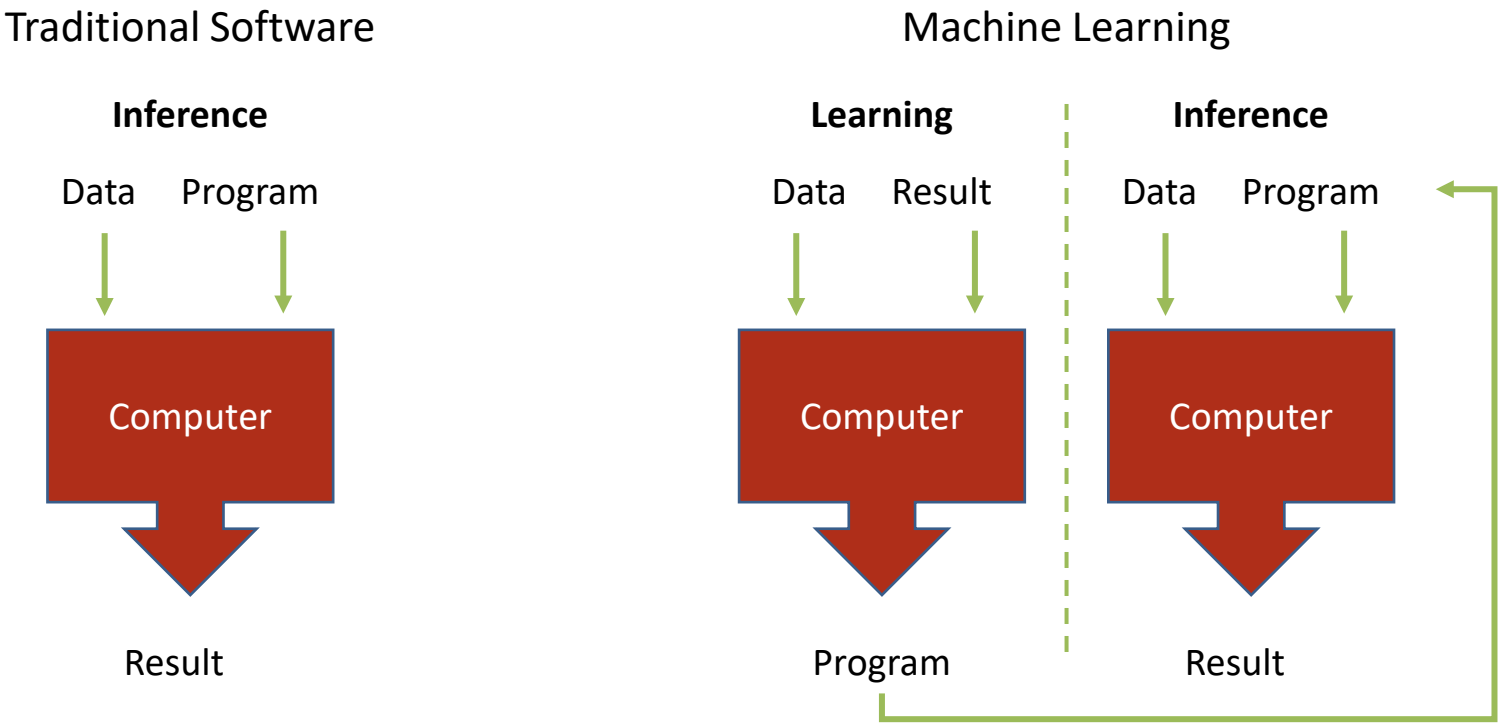
Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

What is Machine Learning (ML)?

Machine Learning:
Generating a model based on inputs (training) and using it for making predictions (productive application)

(Instead of programming instructions explicitly)

Machine Learning vs Traditional Software Systems

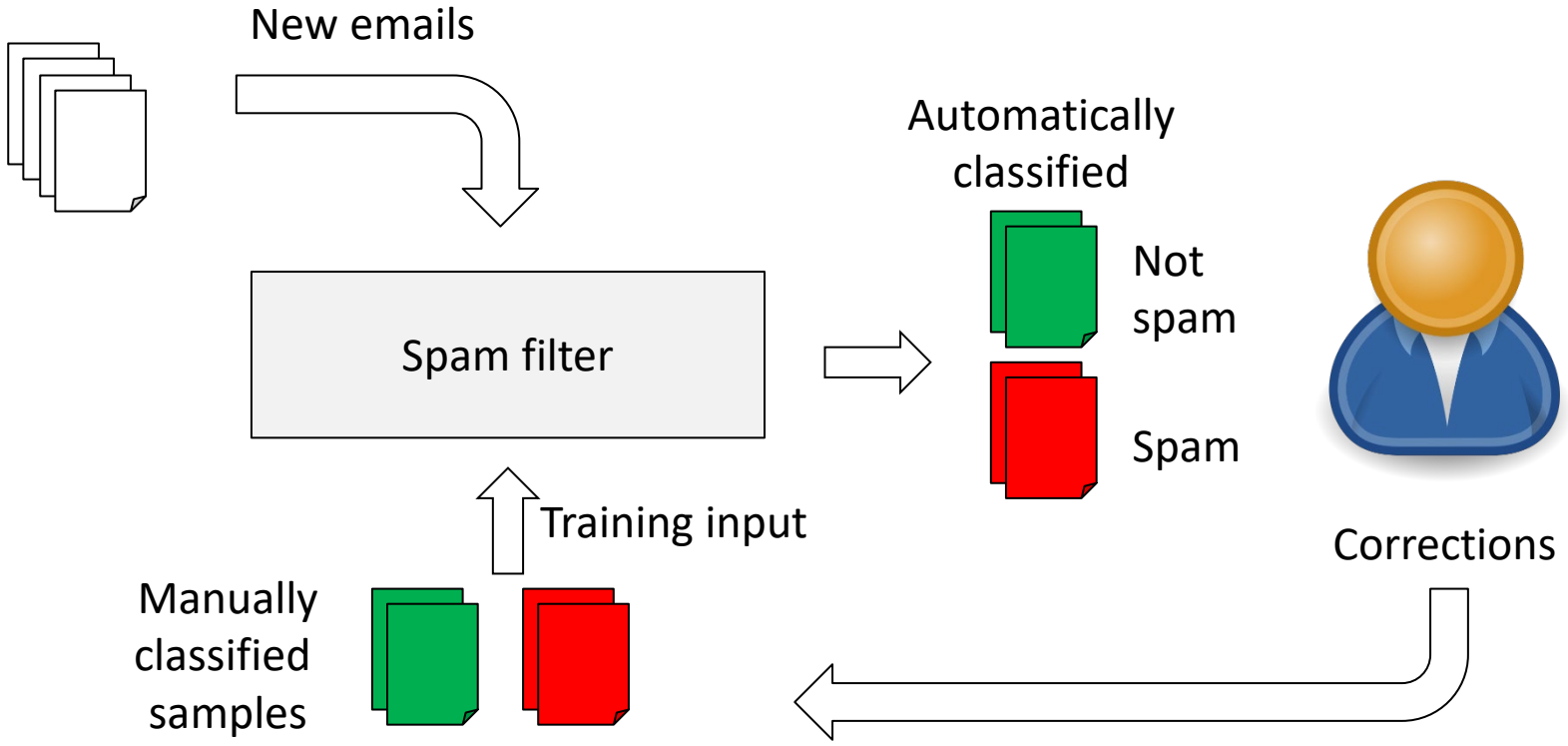


Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Overview of dataset
- Loading data
- Exploring dataset
- Preprocessing data
- ML training
- Predicting
- Quick check

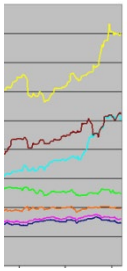
Applications of ML: Spam filtering

- Task: classify new e-mails as spam or not spam

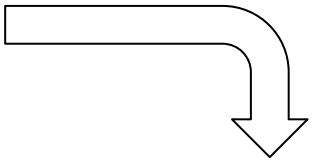


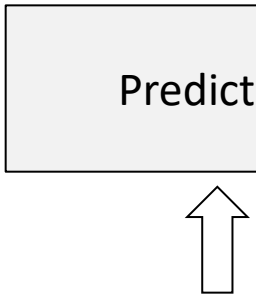
Stock market analysis

- Task: make recommendations on buying and selling stocks



Current share values

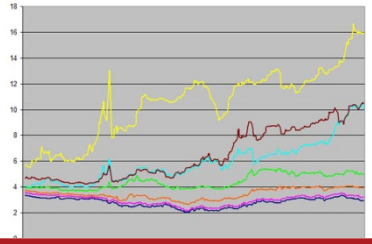






Decision

History of share values



Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

12.09.2024

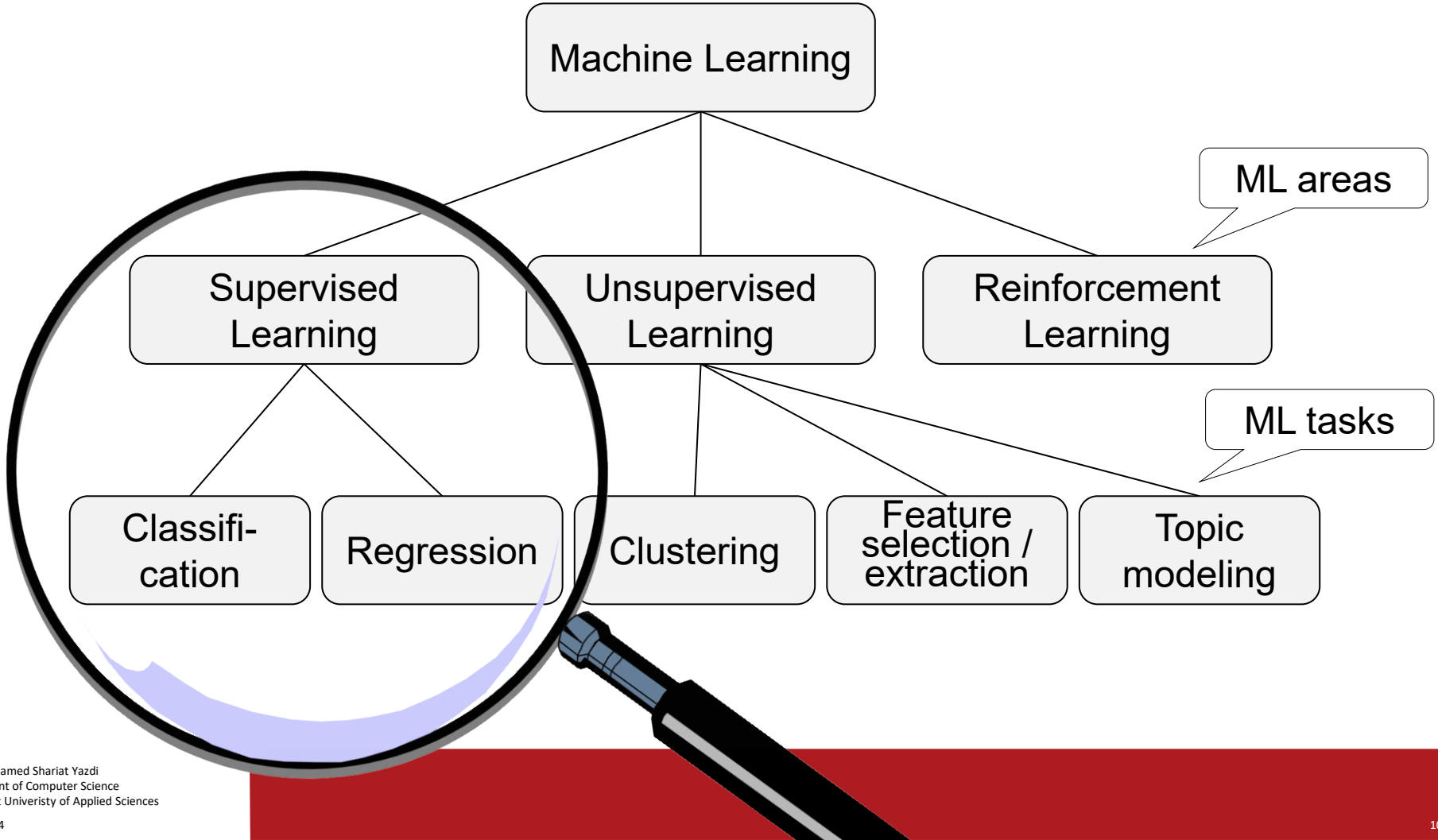
Image source: Wikimedia

Agenda

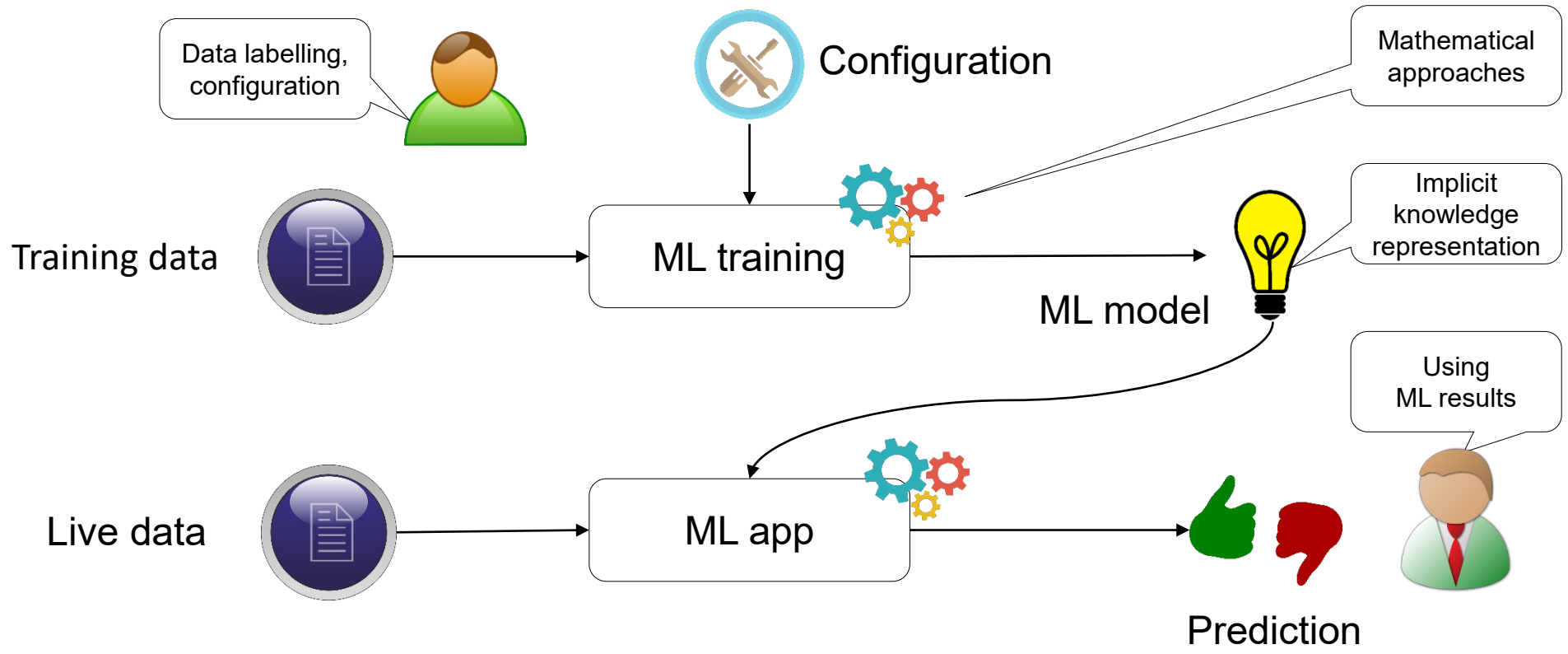
- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Overview of dataset
- Loading data
- Exploring dataset
- Preprocessing data
- ML training
- Predicting
- Quick check

Categories of ML tasks

(Other categorizations / groupings are possible)



Supervised ML



Supervised learning tasks



Classification

- Given: Training inputs (records) which are divided into two or more classes (categories)
- Goal: Produce model to classify new inputs
- Examples: spam filter, fraud detection, …

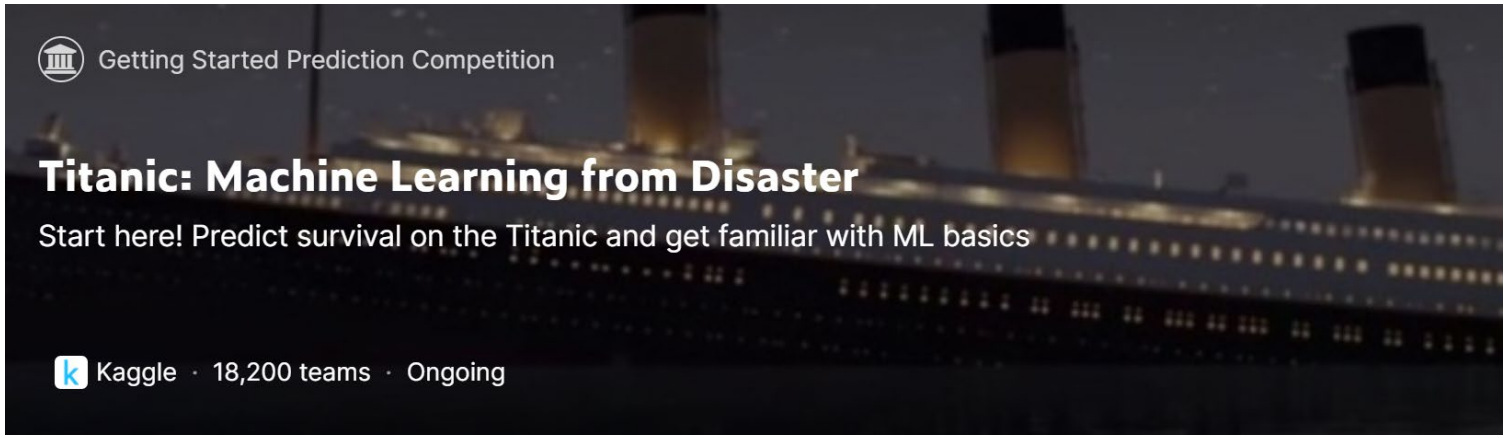
Regression

- Given: Training data (records) with continuous (non categorical) numeric output values
- Goal: Produce model to predict output values for new inputs
- Example: stock value prediction

Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Overview of dataset
- Loading data
- Exploring dataset
- Preprocessing data
- ML training
- Predicting
- Quick check

Example:
Predicting survival on the Titanic



- „Hello World“ example for ML
- Dozens of tutorials available, e.g., https://www.kaggle.com/alexisbcook/titanic-tutorial
- Input (“features”): information about passengers, e.g., gender, age
- Output (“label” / “target” classes): Survived / died

https://www.kaggle.com/c/titanic



The Challenge

The sinking of the Titanic is one of the most infamous shipwrecks in history.

On April 15, 1912, during her maiden voyage, the widely considered “unsinkable” RMS Titanic sank after colliding with an iceberg. Unfortunately, there weren’t enough lifeboats for everyone onboard, resulting in the death of 1502 out of 2224 passengers and crew.

While there was some element of luck involved in surviving, it seems some groups of people were more likely to survive than others.

In this challenge, we ask you to build a predictive model that answers the question: “what sorts of people were more likely to survive?” using passenger data (ie name, age, gender, socio-economic class, etc).

https://www.kaggle.com/c/titanic

What I don’t like about the Titanic example

- It is not a realistic ML application
- Why?

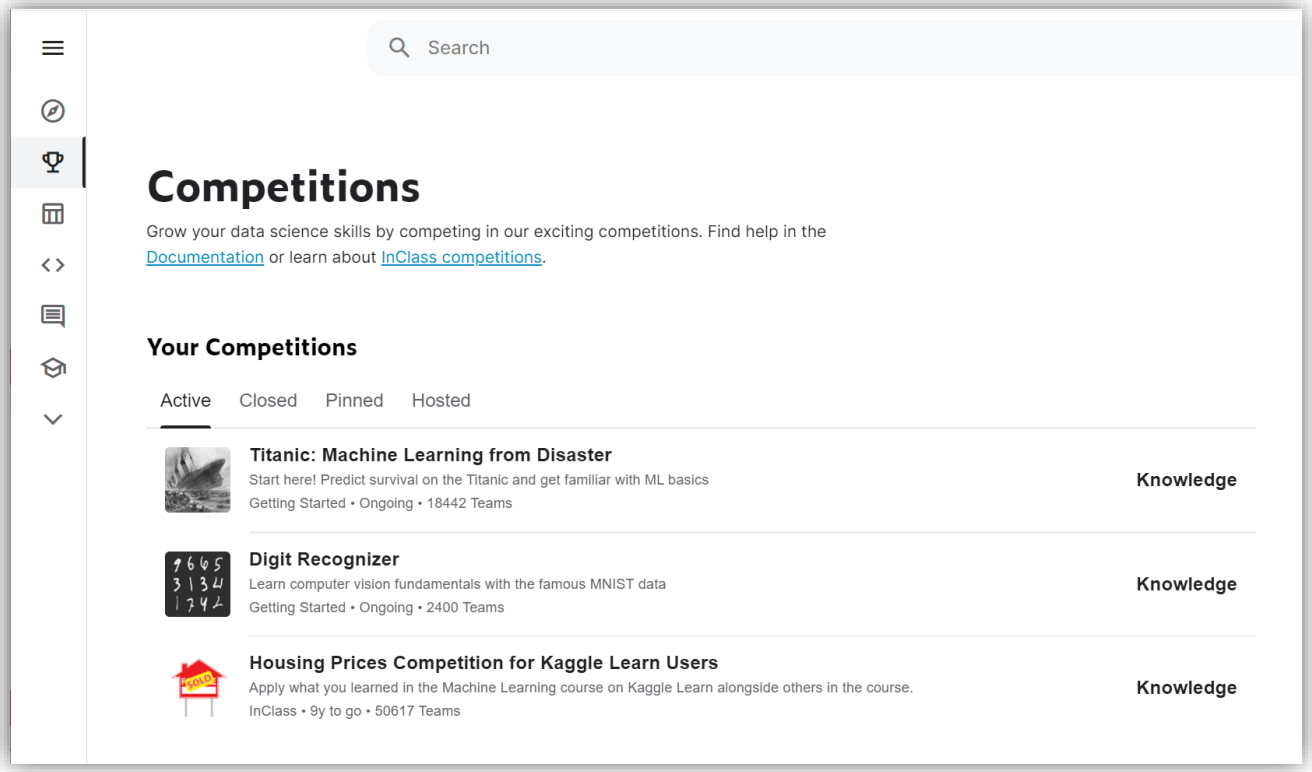


https://cdn.pixabay.com/photo/2015/11/03/08/56/question-mark-1019820_960_720.jpg

Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Overview of dataset
- Loading data
- Exploring dataset
- Preprocessing data
- ML training
- Predicting
- Quick check

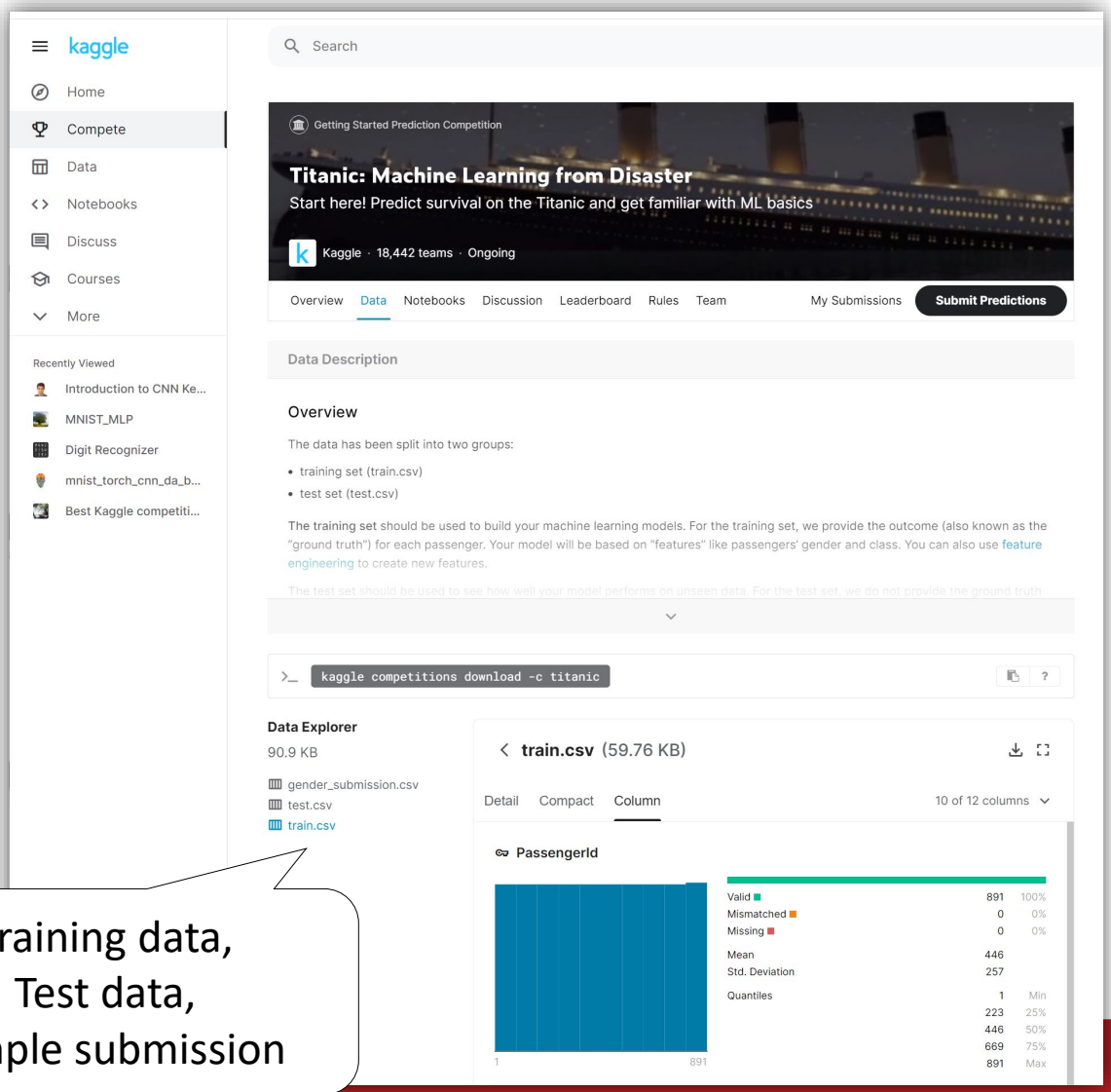
Kaggle: Data science community with datasets, competitions, boards etc.



https://www.kaggle.com

Download data from Kaggle

https://www.kaggle.com/c/titanic/data



Training data,
Test data,
Sample submission

Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Overview of dataset
- Loading data
- Exploring dataset
- Preprocessing data
- ML training
- Predicting
- Quick check

A look into the dataset

	A	B	C	D	E	F	G	H	I	J	K	L
1	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
2	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25		S
3	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38	1	0	PC 17599	71.2833	C85	C
4	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925		S
5	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1	C123	S
6	5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05		S
7	6	0	3	Moran, Mr. James	male		0	0	330877	8.4583		Q
8	7	0	1	McCarthy, Mr. Timothy J	male	54	0	0	17463	51.8625	E46	S
9	8	0	3	Palsson, Master. Gosta Leonard	male	2	3	1	349909	21.075		S
10	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27	0	2	347742	11.1333		S
11	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14	1	0	237736	30.0708		C
12	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4	1	1	PP 9549	16.7	G6	S
13	12	1	1	Bonnell, Miss. Elizabeth	female	58	0	0	113783	26.55	C103	S
14	13	0	3	Saundercock, Mr. William Henry	male	20	0	0	A/5. 2151	8.05		S
15	14	0	3	Andersson, Mr. Anders Johan	male	39	1	5	347082	31.275		S
16	15	0	3	Vestrom, Miss. Hulda Amanda Adolfina	female	14	0	0	350406	7.8542		S
17	16	1	2	Hewlett, Mrs. (Mary D Kingcome)	female	55	0	0	248706	16		S
18	17	0	3	Rice, Master. Eugene	male	2	4	1	382652	29.125		Q
19	18	1	2	Williams, Mr. Charles Eugene	male		0	0	244373	13		S
20	19	0	3	Vander Planke, Mrs. Julius (Emelia Maria Vandemoortele	female	31	1	0	345763	18		S
21	20	1	3	Masselmani, Mrs. Fatima	female		0	0	2649	7.225		C
22	21	0	2	Fynney, Mr. Joseph J	male	35	0	0	239865	26		S
23	22	1	2	Beesley, Mr. Lawrence	male	34	0	0	248698	13	D56	S
24	23	1	3	McGowan, Miss. Anna "Annie"	female	15	0	0	330923	8.0292		Q
25	24	1	1	Sloper, Mr. William Thompson	male	28	0	0	113788	35.5	A6	S
26	25	0	3	Palsson, Miss. Torborg Danira	female	8	3	1	349909	21.075		S
27	26	1	3	Asplund, Mrs. Carl Oscar (Selma Augusta Emilia Johanss	female	38	1	5	347077	31.3875		S
28	27	0	3	Emir, Mr. Farred Chehab	male		0	0	2631	7.225		C
29	28	0	1	Fortune, Mr. Charles Alexander	male	19	3	2	19950	263	C23 C25 C27	S

Attributes of the Titanic dataset

Label
(ML output)

Features
(ML input)

Unique Id, not used for classification

"Label" Classification Target

Passenger class

#siblings and spouse

#parents and children

Port of embarkation

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
689	690	1	1	Madill, Miss. Georgette Alexandra	female	15.0	0	1	24160	211.3375	B5	S
525	526	0	3	Farrell, Mr. James	male	40.5	0	0	367232	7.7500	NaN	Q
278	279	0	3	Rice, Master. Eric	male	7.0	4	1	382652	29.1250	NaN	Q

0 = Died
1 = Survived

1st class,
2nd class,
3rd class
Tickets

Age in years

Ticket number

Missing values **NaN**

C = Cherbourg
Q = Queenstown
S = Southampton

https://www.kaggle.com/gunesevitan/titanic-advanced-feature-engineering-tutorial

Attributes of the Titanic dataset (2)

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
689	690	1	1	Madill, Miss. Georgette Alexandra	female	15.0	0	1	24160	211.3375	B5	S
525	526	0	3	Farrell, Mr. James	male	40.5	0	0	367232	7.7500	NaN	Q
278	279	0	3	Rice, Master. Eric	male	7.0	4	1	382652	29.1250	NaN	Q

Category	Categorical Binary	Categorical Ordinal	Textual	Categorical	Numerical	Numerical	Numerical	Textual	Numerical	Textual	Categorical
Datatype	int	int	string	string	float	int	int	int	float	string	string
Missing Values?	no	no	no	no	yes	no	no	no	yes	yes	yes

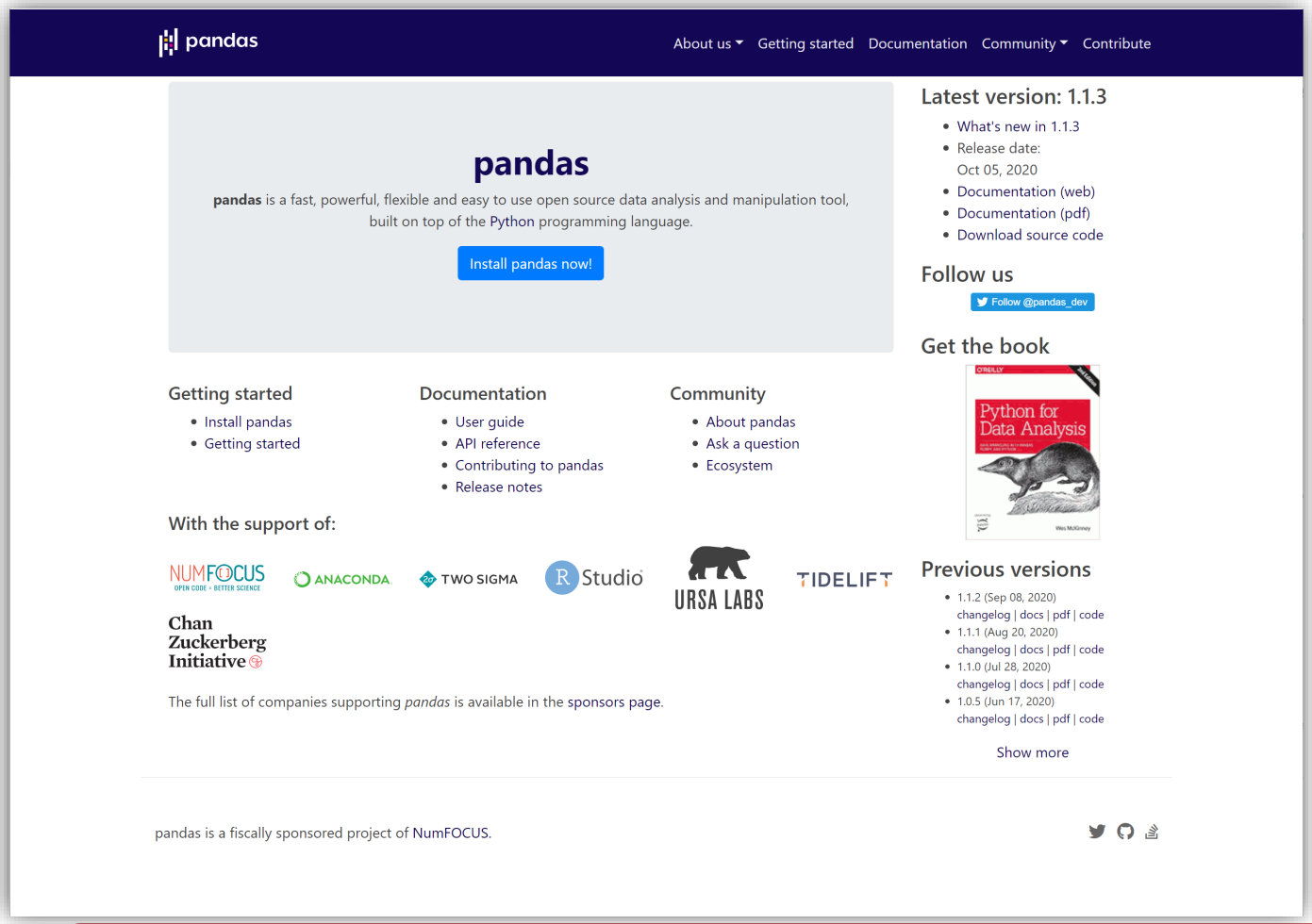
https://www.kaggle.com/gunesevitan/titanic-advanced-feature-engineering-tutorial

Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Overview of dataset
- Loading data
- Exploring dataset
- Preprocessing data
- ML training
- Predicting
- Quick check

Data analysis package: pandas

```
pip install pandas
```



Pandas basics cheat sheet

datacamp

Python For Data Science

Pandas Basics Cheat Sheet

Learn Pandas Basics online at www.DataCamp.com

Pandas

The **Pandas** library is built on NumPy and provides easy-to-use **data structures** and **data analysis** tools for the Python programming language.

Use the following import convention:

```
>>> import pandas as pd
```

> Pandas Data Structures

Series

A **one-dimensional** labeled array capable of holding any data type

Index →

a	3
b	-5
c	7
d	4

```
>>> s = pd.Series([3, -5, 7, 4], index=['a', 'b', 'c', 'd'])
```

Dataframe

A **two-dimensional** labeled data structure with columns of potentially different types

Columns →

Index →	Country	Capital	Population
0	Belgium	Brussels	11190846
1	India	New Delhi	1303171035
2	Brazil	Brasilia	207847528

Axis 0 ↓ Axis 1 →

```
>>> data = {'Country': ['Belgium', 'India', 'Brazil'],
            'Capital': ['Brussels', 'New Delhi', 'Brasilia'],
            'Population': [11190846, 1303171035, 207847528]}
>>> df = pd.DataFrame(data,
                      columns=['Country', 'Capital', 'Population'])
```

> Dropping

```
>>> s.drop(['a', 'c']) #Drop values from rows (axis=0)
>>> df.drop('Country', axis=1) #Drop values from columns(axis=1)
```

> Asking For Help

```
>>> help(pd.Series.loc)
```

> Sort & Rank

```
>>> df.sort_index() #Sort by labels along an axis
>>> df.sort_values(by='Country') #Sort by the values along an axis
>>> df.rank() #Assign ranks to entries
```

> I/O

Read and Write to CSV

```
>>> pd.read_csv('file.csv', header=None, nrows=5)
>>> df.to_csv('myDataFrame.csv')
```

Read and Write to Excel

```
>>> pd.read_excel('file.xlsx')
>>> df.to_excel('dir/myDataFrame.xlsx', sheet_name='Sheet1')
```

Read multiple sheets from the same file

```
>>> xlsx = pd.ExcelFile('file.xls')
>>> df = pd.read_excel(xlsx, 'Sheet1')
```

Read and Write to SQL Query or Database Table

```
>>> from sqlalchemy import create_engine
>>> engine = create_engine('sqlite:///:memory:')
>>> pd.read_sql("SELECT * FROM my_table;", engine)
>>> pd.read_sql_table('my_table', engine)
>>> pd.read_sql_query("SELECT * FROM my_table;", engine)
```

read_sql() **is a convenience wrapper around** read_sql_table() **and** read_sql_query()

```
>>> df.to_sql('myDf', engine)
```

> Selection

Also see NumPy Arrays

Getting

```
>>> s['b'] #Get one element
 -5
>>> df[1:] #Get subset of a DataFrame
   Country Capital Population
 1 India New Delhi 1303171035
 2 Brazil Brasilia 207847528
```

Selecting, Boolean Indexing & Setting

By Position

```
>>> df.iloc[[0],[0]] #Select single value by row & column
 'Belgium'
>>> df.iat[[0],[0])
 'Belgium'
```

By Label

```
>>> df.loc[[0], ['Country']] #Select single value by row & column labels
 'Belgium'
>>> df.at([0], ['Country'])
 'Belgium'
```

By Label/Position

```
>>> df.ix[2] #Select single row of subset of rows
 Country Brazil
 Capital Brasilia
 Population 207847528
>>> df.ix[:,'Capital'] #Select a single column of subset of columns
 0 Brussels
 1 New Delhi
 2 Brasilia
>>> df.ix[1,'Capital'] #Select rows and columns
 'New Delhi'
```

Boolean Indexing

```
>>> s[~(s > 1)] #Series s where value is not >1
>>> s[(s < -1) | (s > 2)] #s where value is <-1 or >2
>>> df[df['Population']>1200000000] #Use filter to adjust DataFrame
```

Setting

```
>>> s['a'] = 6 #Set index a of Series s to 6
```

> Retrieving Series/DataFrame Information

Basic Information

```
>>> df.shape #(rows,columns)
>>> df.index #Describe index
>>> df.columns #Describe DataFrame columns
>>> df.info() #Info on DataFrame
>>> df.count() #Number of non-NA values
```

Summary

```
>>> df.sum() #Sum of values
>>> df.cumsum() #Cummulative sum of values
>>> df.min()/df.max() #Minimum/maximum values
>>> df.idxmin()/df.idxmax() #Minimum/Maximum index value
>>> df.describe() #Summary statistics
>>> df.mean() #Mean of values
>>> df.median() #Median of values
```

> Applying Functions

```
>>> f = lambda x: x*2
>>> df.apply(f) #Apply function
>>> df.applymap(f) #Apply function element-wise
```

> Data Alignment

Internal Data Alignment

NA values are introduced in the indices that don't overlap:

```
>>> s3 = pd.Series([7, -2, 3], index=['a', 'c', 'd'])
>>> s + s3
 a 10.0
 b NaN
 c 5.0
 d 7.0
```

Arithmetic Operations with Fill Methods

You can also do the internal data alignment yourself with the help of the fill methods:

```
>>> s.add(s3, fill_values=0)
 a 10.0
 b -5.0
 c 5.0
 d 7.0
>>> s.sub(s3, fill_value=2)
>>> s.div(s3, fill_value=4)
>>> s.mul(s3, fill_value=3)
```



https://www.datacamp.com/community/blog/python-pandas-cheat-sheet#gs.oundfxM

Loading DataFrame from CSV file

```
import pandas as pd
```

```
train_data = pd.read_csv('data/train.csv')
test_data = pd.read_csv('data/test.csv')
```

Reads CSV file (relative location to notebook file) and stores it in a **pd.DataFrame**.

Optional parameters, e.g., encoding, delimiter, quotechar , …

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html

pd.DataFrame
Primary pandas data structure

pandas.DataFrame

class pandas.DataFrame(*data=None, index=None, columns=None, dtype=None, copy=False*) [source]

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.

Parameters:

data : *ndarray (structured or homogeneous), Iterable, dict, or DataFrame*

Dict can contain Series, arrays, constants, or list-like objects.

Changed in version 0.23.0: If data is a dict, column order follows insertion-order for Python 3.6 and later.

Changed in version 0.25.0: If data is a list of dicts, column order follows insertion-order for Python 3.6 and later.

index : *Index or array-like*

Index to use for resulting frame. Will default to RangeIndex if no indexing information part of input data and no index provided.

columns : *Index or array-like*

Column labels to use for resulting frame. Will default to RangeIndex (0, 1, 2, …, n) if no column labels are provided.

dtype : *dtype, default None*

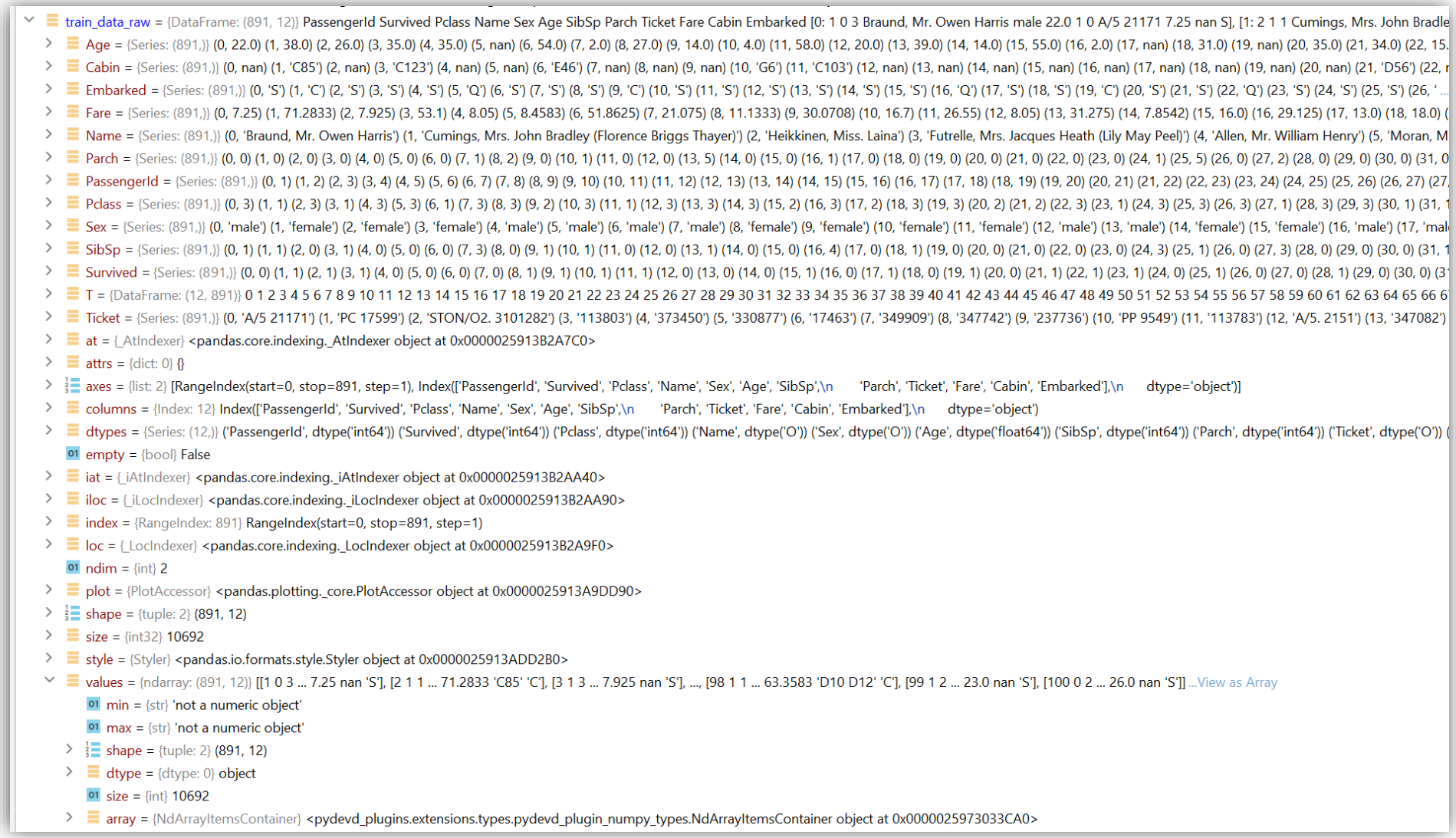
Data type to force. Only a single dtype is allowed. If None, infer.

copy : *bool, default False*

Copy data from inputs. Only affects DataFrame / 2d ndarray input.

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.html

Inspecting the DataFrame

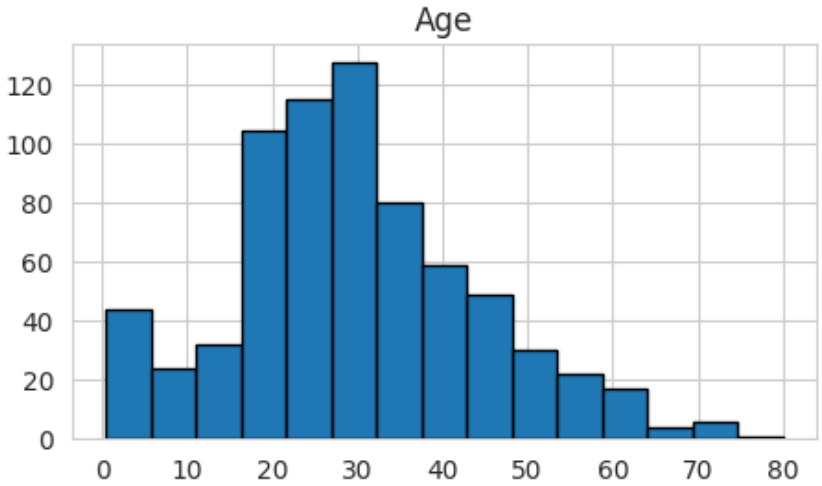


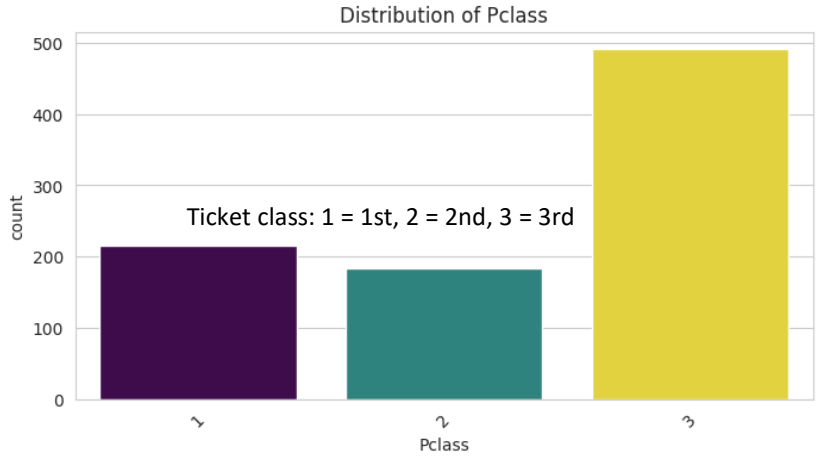
Agenda

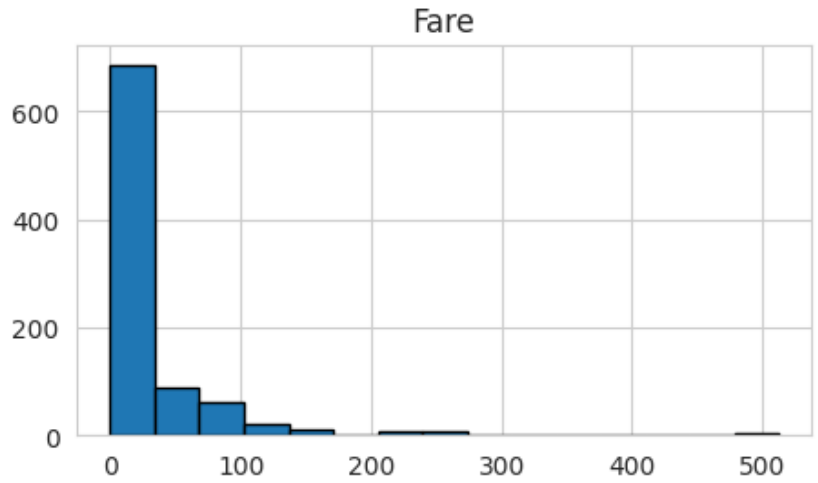
- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Overview of dataset
- Loading data
- Exploring dataset
- Preprocessing data
- ML training
- Predicting
- Quick check

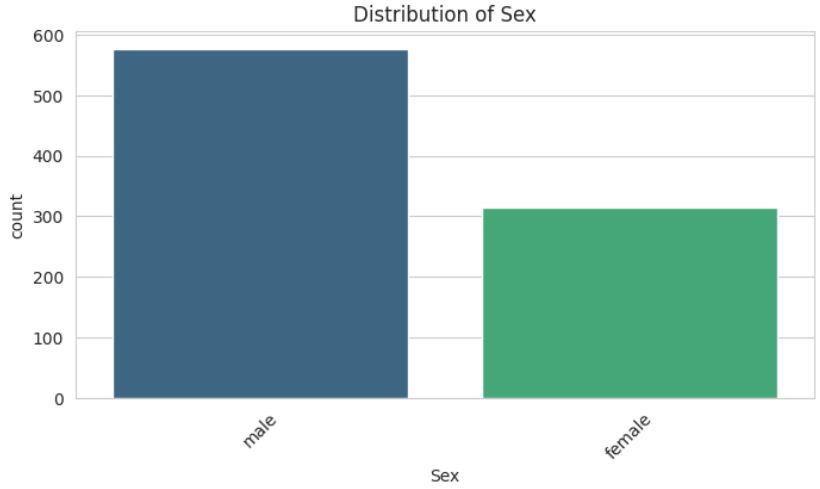
Exploring dataset

Distribution of Numerical and Categorical Data



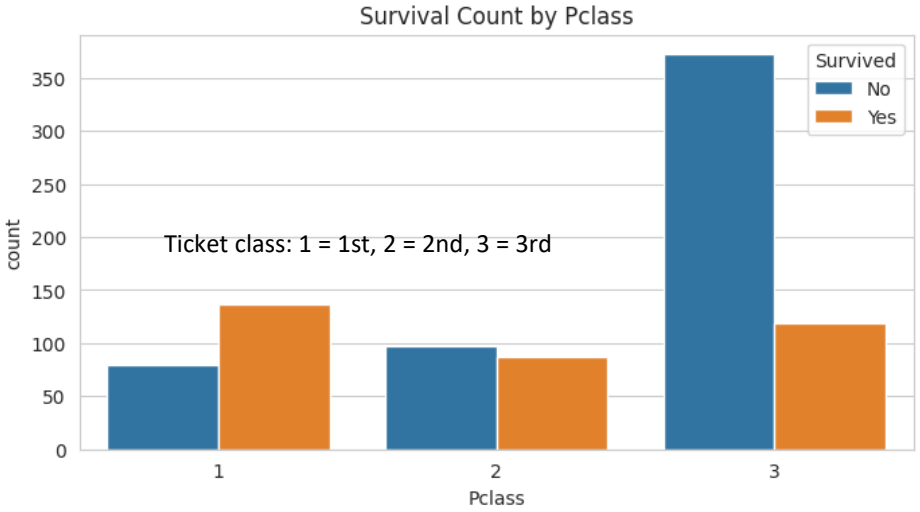


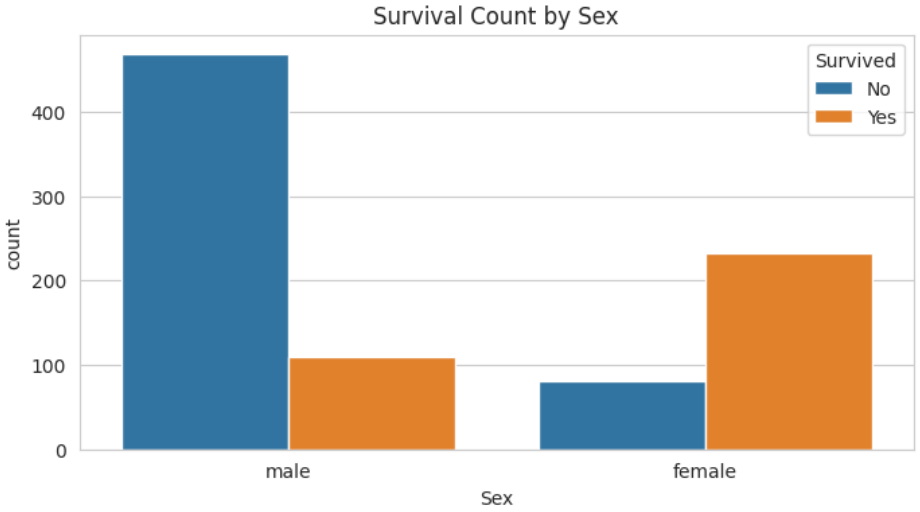


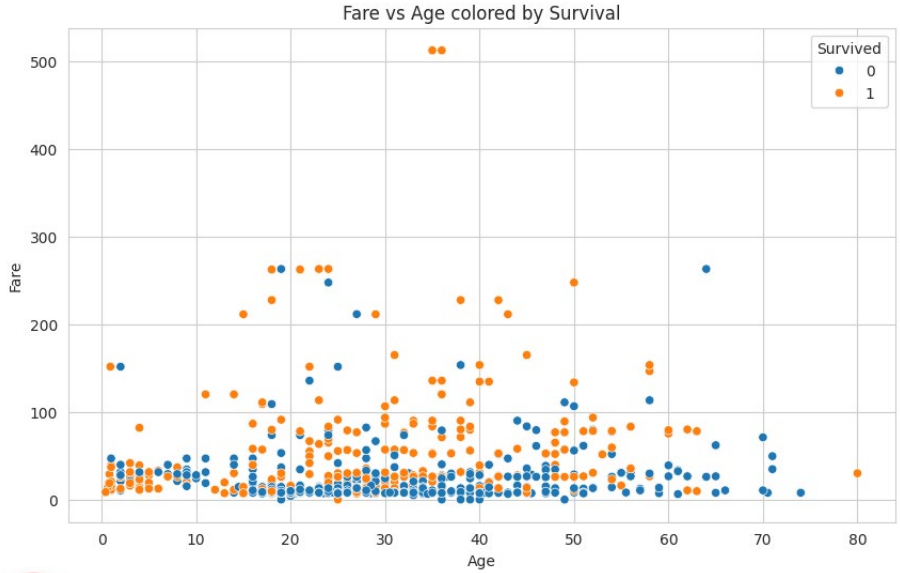


Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Exploring dataset









Can we have a model to predict the survival?

- 1st class passengers survived more.
- Female survived more.
- Those paid more survived more.

Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Overview of dataset
- Loading data
- Exploring dataset
- Preprocessing data
- ML training
- Predicting
- Quick check

Pre-procesing data: Extracting relevant columns

```
# extract labels
y = train_data ["Survived"]

# select relevant features (maybe extended later)
features = ["Pclass", "Sex", "SibSp", "Parch"]
X = train_data [features]
```

Extracting columns from a dataset
One column name → vector
List of column names → matrix

Naming convention from mathematics:
Matrix: uppercase, e.g., X
Vector: lowercase, e.g., y
Input: x (or X)
Output: y (or Y)

Additional pre-processing: Filling missing values

DataFrame.fillna returns modified DataFrame (without modifying the original DataFrame)
Assign it if you want to change it

```
# fill missings (n/a values) with fixed value
data['Embarked'] = data['Embarked'].fillna('S')
```

Use **SimpleImputer** for different filling strategies

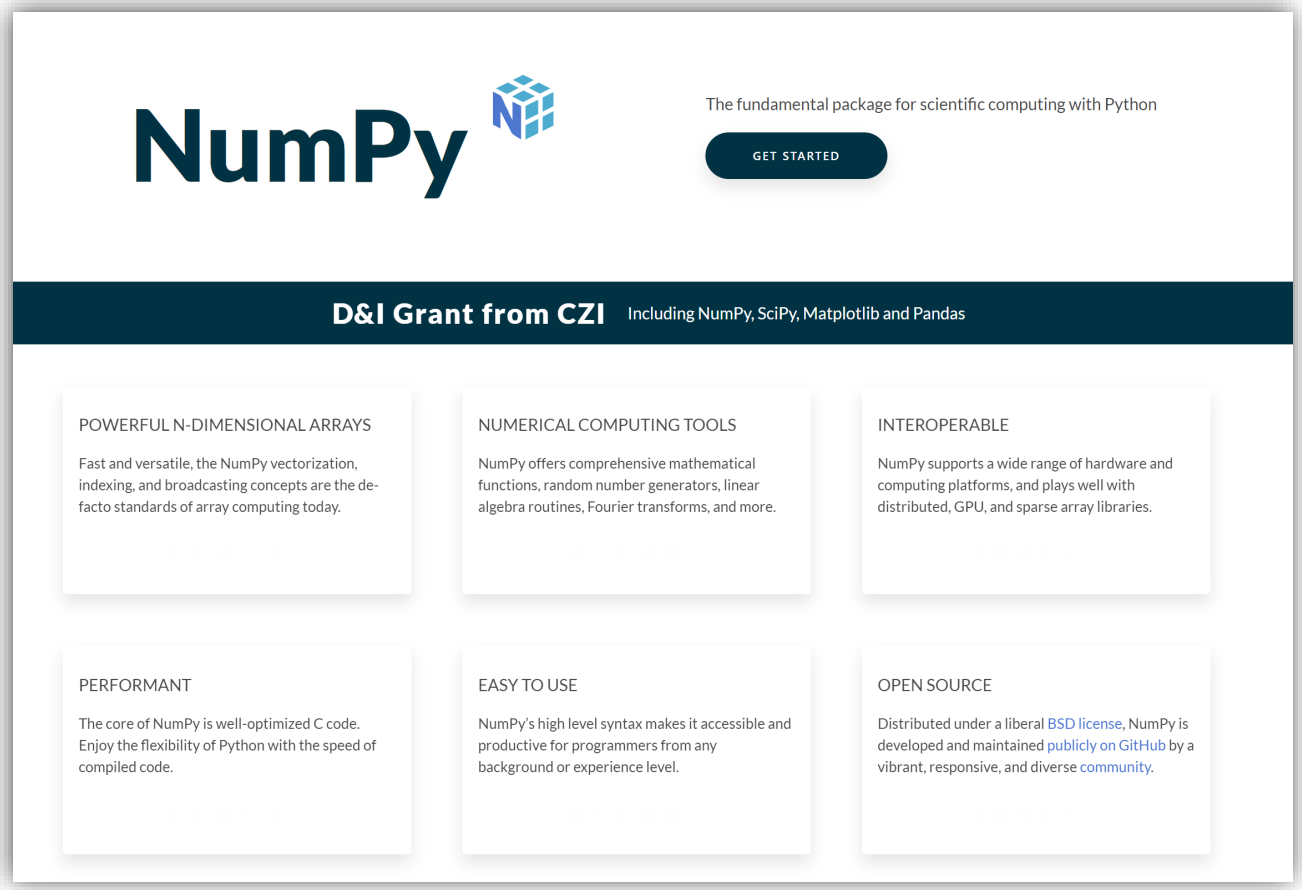
```
# fill missings with mean of all values
import numpy as np
from sklearn.impute import SimpleImputer

imp = SimpleImputer(missing_values=np.nan, strategy='mean')
data['Age'] = imp.fit_transform(data[['Age']])
```

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.fillna.html
https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html

Scientific computing package: NumPy

```
pip install numpy
```



https://numpy.org

Numpy basics cheat sheet

datacamp

Python For Data Science NumPy Cheat Sheet

Learn NumPy online at www.DataCamp.com

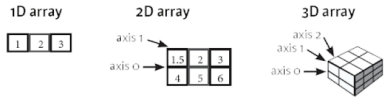
Numpy

The NumPy library is the core library for scientific computing in Python. It provides a high-performance multidimensional array object, and tools for working with these arrays

Use the following import convention:

```
>>> import numpy as np
```

NumPy Arrays



> Creating Arrays

```
>>> a = np.array([1,2,3])
>>> b = np.array([(1.5,2,3), (4,5,6)], dtype = float)
>>> c = np.array([[(1.5,2,3), (4,5,6)],[(3,2,1), (4,5,6)]], dtype = float)
```

Initial Placeholders

```
>>> np.zeros((3,4)) #Create an array of zeros
>>> np.ones((2,3,4),dtype=np.int16) #Create an array of ones
>>> d = np.arange(10,25,5) #Create an array of evenly spaced values (step value)
>>> np.linspace(0,2,9) #Create an array of evenly spaced values (number of samples)
>>> e = np.full((2,2),7) #Create a constant array
>>> f = np.eye(2) #Create a 2X2 identity matrix
>>> np.random.random((2,2)) #Create an array with random values
>>> np.empty((3,2)) #Create an empty array
```

> I/O

Saving & Loading On Disk

```
>>> np.save('my_array', a)
>>> np.savez('array.npz', a, b)
>>> np.load('my_array.npy')
```

Saving & Loading Text Files

```
>>> np.loadtxt("myfile.txt")
>>> np.genfromtxt("my_file.csv", delimiter=',')
>>> np.savetxt("myarray.txt", a, delimiter=" ")
```

> Asking For Help

```
>>> np.info(np.ndarray.dtype)
```

> Inspecting Your Array

```
>>> a.shape #Array dimensions
>>> len(a) #Length of array
>>> b.ndim #Number of array dimensions
>>> e.size #Number of array elements
>>> b.dtype #Data type of array elements
>>> b.dtype.name #Name of data type
>>> b.astype(int) #Convert an array to a different type
```

> Data Types

```
>>> np.int64 #Signed 64-bit integer types
>>> np.float32 #Standard double-precision floating point
>>> np.complex #Complex numbers represented by 128 floats
>>> np.bool #Boolean type storing TRUE and FALSE values
>>> np.object #Python object type
>>> np.string_ #Fixed-length string type
>>> np.unicode_ #Fixed-length unicode type
```

> Array Mathematics

Arithmetic Operations

```
>>> g = a - b #Subtraction
 array([[-0.5, 0. , 0. ],
        [-3. , -3. , -3. ]])
>>> np.subtract(a,b) #Subtraction
>>> b + a #Addition
 array([[ 2.5, 4. , 6. ],
        [ 5. , 7. , 9. ]])
>>> np.add(b,a) #Addition
>>> a / b #Division
 array([[ 0.66666667, 1. , 1. ],
        [ 0.25 , 0.4 , 0.5 ]])
>>> np.divide(a,b) #Division
>>> a * b #Multiplication
 array([[ 1.5, 4. , 9. ],
        [ 4. , 10. , 18. ]])
>>> np.multiply(a,b) #Multiplication
>>> np.exp(b) #Exponentiation
>>> np.sqrt(b) #Square root
>>> np.sin(a) #Print sines of an array
>>> np.cos(b) #Element-wise cosine
>>> np.log(a) #Element-wise natural logarithm
>>> e.dot(f) #Dot product
 array([[ 7., 7.],
        [ 7., 7.]])
```

Comparison

```
>>> a == b #Element-wise comparison
 array([[False, True, True],
        [False, False, False]], dtype=bool)
>>> a < 2 #Element-wise comparison
 array([True, False, False], dtype=bool)
>>> np.array_equal(a, b) #Array-wise comparison
```

Aggregate Functions

```
>>> a.sum() #Array-wise sum
>>> a.min() #Array-wise minimum value
>>> b.max(axis=0) #Maximum value of an array row
>>> b.cumsum(axis=1) #Cumulative sum of the elements
>>> a.mean() #Mean
>>> np.median(b) #Median
>>> np.corrcoef(a) #Correlation coefficient
>>> np.std(b) #Standard deviation
```

> Copying Arrays

```
>>> h = a.view() #Create a view of the array with the same data
>>> np.copy(a) #Create a copy of the array
>>> h = a.copy() #Create a deep copy of the array
```

> Sorting Arrays

```
>>> a.sort() #Sort an array
>>> c.sort(axis=0) #Sort the elements of an array's axis
```

> Subsetting, Slicing, Indexing

Subsetting

```
>>> a[2] #Select the element at the 2nd index
 3
>>> b[1,2] #Select the element at row 1 column 2 (equivalent to b[1][2])
 6.0
```

Slicing

```
>>> a[0:2] #Select items at index 0 and 1
 array([1, 2])
>>> b[0:2,1] #Select items at rows 0 and 1 in column 1
 array([ 2., 5.])
>>> b[:1] #Select all items at row 0 (equivalent to b[0:1, :])
 array([[1.5, 2., 3.]])
>>> c[1,...] #Same as [1,:,:]
 array([[[ 3., 2., 1.],
         [ 4., 5., 6.]]])
>>> a[ : :-1] #Reversed array a array([3, 2, 1])
```

Boolean Indexing

```
>>> a[a<2] #Select elements from a less than 2
 array([1])
```

Fancy Indexing

```
>>> b[[1, 0, 1, 0],[0, 1, 2, 0]] #Select elements (1,0),(0,1),(1,2) and (0,0)
 array([ 4. , 2. , 6. , 1.5])
>>> b[[1, 0, 1, 0]][:,[0,1,2,0]] #Select a subset of the matrix's rows and columns
 array([[ 4. ,5. , 6. , 4. ],
        [ 1.5, 2. , 3. , 1.5],
        [ 4. , 5. , 6. , 4. ],
        [ 1.5, 2. , 3. , 1.5]])
```

> Array Manipulation

Transposing Array

```
>>> i = np.transpose(b) #Permute array dimensions
>>> i.T #Permute array dimensions
```

Changing Array Shape

```
>>> b.ravel() #Flatten the array
>>> g.reshape(3,-2) #Reshape, but don't change data
```

Adding/Removing Elements

```
>>> h.resize((2,6)) #Return a new array with shape (2,6)
>>> np.append(h,g) #Append items to an array
>>> np.insert(a, 1, 5) #Insert items in an array
>>> np.delete(a,[1]) #Delete items from an array
```

Combining Arrays

```
>>> np.concatenate((a,d),axis=0) #Concatenate arrays
 array([ 1, 2, 3, 10, 15, 20])
>>> np.vstack((a,b)) #Stack arrays vertically (row-wise)
 array([[ 1. , 2. , 3. ],
        [ 1.5, 2. , 3. ],
        [ 4. , 5. , 6. ]])
>>> np.r_[e,f] #Stack arrays vertically (row-wise)
>>> np.hstack((e,f)) #Stack arrays horizontally (column-wise)
 array([[ 7., 7., 1., 0.],
        [ 7., 7., 0., 1.]])
>>> np.column_stack((a,d)) #Create stacked column-wise arrays
 array([[ 1, 10],
        [ 2, 15],
        [ 3, 20]])
>>> np.c_[a,d] #Create stacked column-wise arrays
```

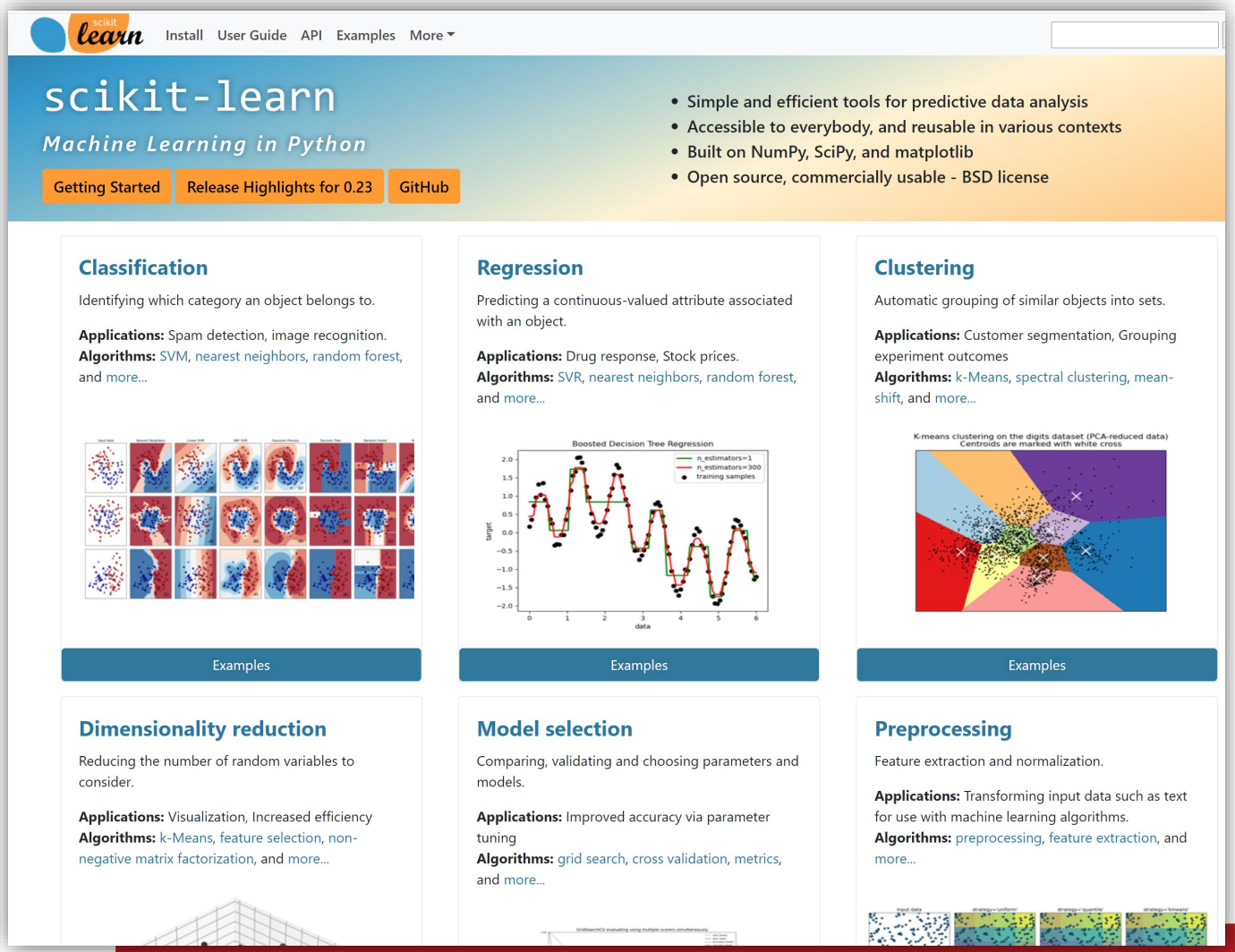
Splitting Arrays

```
>>> np.hsplit(a,3) #Split the array horizontally at the 3rd index
 [array([1]),array([2]),array([3])]
>>> np.vsplit(c,2) #Split the array vertically at the 2nd index
 [array([[[ 1.5, 2. , 1. ],
          [ 4. , 5. , 6. ]]]),
 array([[[ 3., 2., 3.],
         [ 4., 5., 6.]]])]
```

Learn Data Skills Online at www.DataCamp.com

ML library scikit-learn

```
pip install scikit-learn
```



https://scikit-learn.org

Scikit-learn cheat sheet



datacamp

Python For Data Science

Scikit-Learn Cheat Sheet

Learn Scikit-Learn online at www.DataCamp.com

Scikit-learn

Scikit-learn is an open source Python library that implements a range of machine learning, preprocessing, cross-validation and visualization algorithms using a unified interface.

A Basic Example

```
>>> from sklearn import neighbors, datasets, preprocessing
>>> from sklearn.model_selection import train_test_split
>>> from sklearn.metrics import accuracy_score
>>> iris = datasets.load_iris()
>>> X, y = iris.data[:, :2], iris.target
>>> X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=33)
>>> scaler = preprocessing.StandardScaler().fit(X_train)
>>> X_train = scaler.transform(X_train)
>>> X_test = scaler.transform(X_test)
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
>>> knn.fit(X_train, y_train)
>>> y_pred = knn.predict(X_test)
>>> accuracy_score(y_test, y_pred)
```

> Loading The Data

Also see NumPy & Pandas

Your data needs to be numeric and stored as NumPy arrays or SciPy sparse matrices. Other types that are convertible to numeric arrays, such as Pandas DataFrame, are also acceptable.

```
>>> import numpy as np
>>> X = np.random.random((10,5))
>>> y = np.array(['M','M','F','F','M','F','M','M','F','F','F'])
>>> X[X < 0.7] = 0
```

> Training And Test Data

```
>>> from sklearn.model_selection import train_test_split
>>> X_train, X_test, y_train, y_test = train_test_split(X,
                                                        y,
                                                        random_state=0)
```

> Model Fitting

Supervised learning

```
>>> lr.fit(X, y) #Fit the model to the data
>>> knn.fit(X_train, y_train)
>>> svc.fit(X_train, y_train)
```

Unsupervised Learning

```
>>> k_means.fit(X_train) #Fit the model to the data
>>> pca_model = pca.fit_transform(X_train) #Fit to data, then transform it
```

> Prediction

Supervised Estimators

```
>>> y_pred = svc.predict(np.random.random((2,5))) #Predict labels
>>> y_pred = lr.predict(X_test) #Predict labels
>>> y_pred = knn.predict_proba(X_test) #Estimate probability of a label
```

Unsupervised Estimators

```
>>> y_pred = k_means.predict(X_test) #Predict labels in clustering algos
```

> Preprocessing The Data

Standardization

```
>>> from sklearn.preprocessing import StandardScaler
>>> scaler = StandardScaler().fit(X_train)
>>> standardized_X = scaler.transform(X_train)
>>> standardized_X_test = scaler.transform(X_test)
```

Normalization

```
>>> from sklearn.preprocessing import Normalizer
>>> scaler = Normalizer().fit(X_train)
>>> normalized_X = scaler.transform(X_train)
>>> normalized_X_test = scaler.transform(X_test)
```

Binarization

```
>>> from sklearn.preprocessing import Binarizer
>>> binarizer = Binarizer(threshold=0.0).fit(X)
>>> binary_X = binarizer.transform(X)
```

Encoding Categorical Features

```
>>> from sklearn.preprocessing import LabelEncoder
>>> enc = LabelEncoder()
>>> y = enc.fit_transform(y)
```

Imputing Missing Values

```
>>> from sklearn.preprocessing import Imputer
>>> imp = Imputer(missing_values=0, strategy='mean', axis=0)
>>> imp.fit_transform(X_train)
```

Generating Polynomial Features

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly = PolynomialFeatures(5)
>>> poly.fit_transform(X)
```

> Create Your Model

Supervised Learning Estimators

Linear Regression

```
>>> from sklearn.linear_model import LinearRegression
>>> lr = LinearRegression(normalize=True)
```

Support Vector Machines (SVM)

```
>>> from sklearn.svm import SVC
>>> svc = SVC(kernel='linear')
```

Naive Bayes

```
>>> from sklearn.naive_bayes import GaussianNB
>>> gnb = GaussianNB()
```

KNN

```
>>> from sklearn import neighbors
>>> knn = neighbors.KNeighborsClassifier(n_neighbors=5)
```

Unsupervised Learning Estimators

Principal Component Analysis (PCA)

```
>>> from sklearn.decomposition import PCA
>>> pca = PCA(n_components=0.95)
```

K Means

```
>>> from sklearn.cluster import KMeans
>>> k_means = KMeans(n_clusters=3, random_state=0)
```

> Evaluate Your Model's Performance

Classification Metrics

Accuracy Score

```
>>> knn.score(X_test, y_test) #Estimator score method
>>> from sklearn.metrics import accuracy_score #Metric scoring functions
>>> accuracy_score(y_test, y_pred)
```

Classification Report

```
>>> from sklearn.metrics import classification_report #Precision, recall, f1-score and support
>>> print(classification_report(y_test, y_pred))
```

Confusion Matrix

```
>>> from sklearn.metrics import confusion_matrix
>>> print(confusion_matrix(y_test, y_pred))
```

Regression Metrics

Mean Absolute Error

```
>>> from sklearn.metrics import mean_absolute_error
>>> y_true = [3, -0.5, 2]
>>> mean_absolute_error(y_true, y_pred)
```

Mean Squared Error

```
>>> from sklearn.metrics import mean_squared_error
>>> mean_squared_error(y_test, y_pred)
```

R^2 Score

```
>>> from sklearn.metrics import r2_score
>>> r2_score(y_true, y_pred)
```

Clustering Metrics

Adjusted Rand Index

```
>>> from sklearn.metrics import adjusted_rand_score
>>> adjusted_rand_score(y_true, y_pred)
```

Homogeneity

```
>>> from sklearn.metrics import homogeneity_score
>>> homogeneity_score(y_true, y_pred)
```

V-measure

```
>>> from sklearn.metrics import v_measure_score
>>> metrics.v_measure_score(y_true, y_pred)
```

Cross-Validation

```
>>> from sklearn.cross_validation import cross_val_score
>>> print(cross_val_score(knn, X_train, y_train, cv=4))
>>> print(cross_val_score(lr, X, y, cv=2))
```

> Tune Your Model

Grid Search

```
>>> from sklearn.grid_search import GridSearchCV
>>> params = {"n_neighbors": np.arange(1,3),
              "metric": ["euclidean", "cityblock"]}
>>> grid = GridSearchCV(estimator=knn,
                        param_grid=params)
>>> grid.fit(X_train, y_train)
>>> print(grid.best_score_)
>>> print(grid.best_estimator_.n_neighbors)
```

Randomized Parameter Optimization

```
>>> from sklearn.grid_search import RandomizedSearchCV
>>> params = {"n_neighbors": range(1,5), "weights": ["uniform", "distance"]}
>>> rsearch = RandomizedSearchCV(estimator=knn, param_distributions=params,
                                 cv=4, n_iter=8, random_state=5)
>>> rsearch.fit(X_train, y_train)
>>> print(rsearch.best_score_)
```

Learn Data Skills Online at www.DataCamp.com

https://www.datacamp.com/community/blog/scikit-learn-cheat-sheet

Prof. Dr. Hamed Shariat Yazdi
Department of Computer Science
Darmstadt Univeristy of Applied Sciences

Other ideas for pre-processing?



- **Handling Missing Values**
  - **Impute with Mean, Median, or Mode**: Fill missing numerical values with the mean, median, or mode. Use the mode for categorical features.
- **Scaling and Normalization**
  - **Standardization (Z-score Scaling)**: Transform features to have a mean of 0 and standard deviation of 1, which is useful for algorithms sensitive to feature scaling (e.g., SVM, linear regression).
  - **Min-Max Scaling**: Scale features to a specific range, typically [0, 1], for algorithms like KNN and neural networks.
- **Encoding Categorical Variables**
  - **One-Hot Encoding**: Convert categorical features into binary columns (0/1) for each category. This is useful for nominal categorical features.
  - **Label Encoding**: Assign unique integer values to categories, which can be useful for ordinal features.
- **Handling Outliers**
  - **Cap and Floor Values**: Limit extreme values by capping at the 5th and 95th percentiles, or at specific threshold values.
  - **Log Transformation**: Apply a log or square root transformation to reduce the impact of outliers in skewed data.
  - **Remove Outliers**: If outliers represent errors or extreme noise, consider removing them.
- **Data Transformation**
  - **Log Transformation**: Useful for features with highly skewed distributions, like income or sales data.

https://cdn.pixabay.com/photo/2015/11/03/08/56/question-mark-1019820_960_720.jpg

Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Analyzing dataset
- Loading data
- Preprocessing data
- ML training
- Predicting
- Quick check

ML training

```
# create ML model
model = DecisionTreeClassifier ()

# train ML model
model.fit (X, y)
```

ML approach chosen: DecisionTreeClassifier

Training (“fitting“) the ML model with features (X) and labels (y)

https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

Decision Tree Learning

- Decision tree: tree with classes as leaf nodes (here: died, survived), decisions as inner nodes (e.g., is sex male?) and values as edges (e.g., yes, no)
- Decision tree learning: automatically generating decision tree from training data (features and labels)
- Example: Decision tree for Titanic survival classification

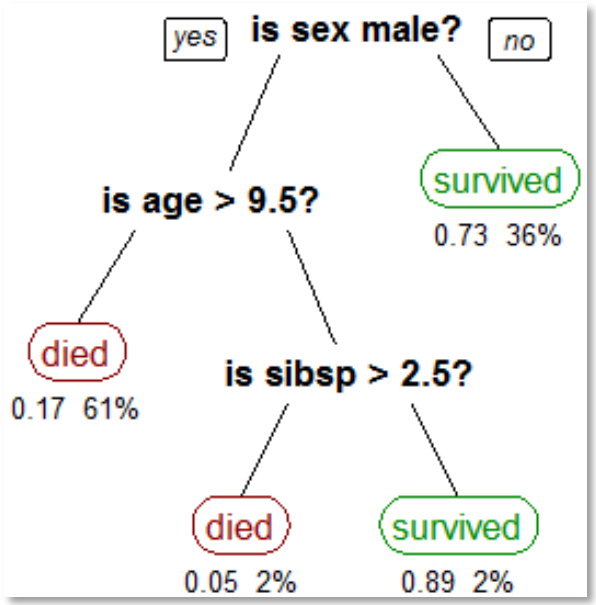
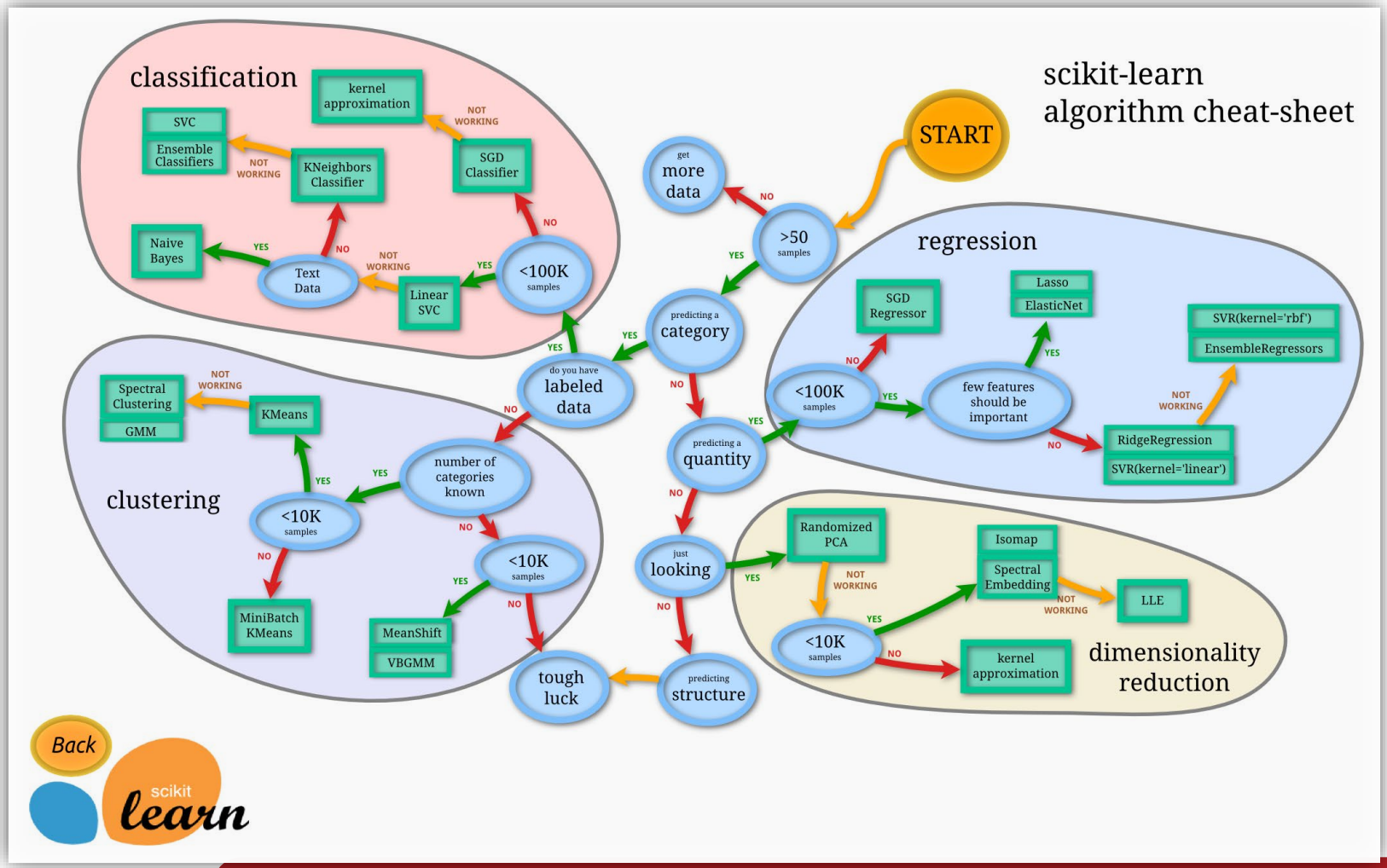


Image source: Wikipedia

Ideas for other ML approaches: ML cheat sheet



Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Analyzing dataset
- Loading data
- Preprocessing data
- ML training
- Predicting
- Quick check

Use ML model for predicting new cases

```
# predict labels for test set
predictions = model.predict(X_test)
```

Predicts all cases in test set at once

Important: Test set must have been preprocessed like training set

Submission format required by Kaggle

	A	B	C
1	PassengerId	Survived	
2	892	0	
3	893	0	
4	894	1	
5	895	1	
6	896	1	
7	897	0	
8	898	0	
9	899	0	
10	900	1	
11	901	0	
12	902	0	
13	903	0	
14	904	1	
15	905	0	
16	906	1	
17	907	1	
18	908	0	
19	909	1	
20	910	1	

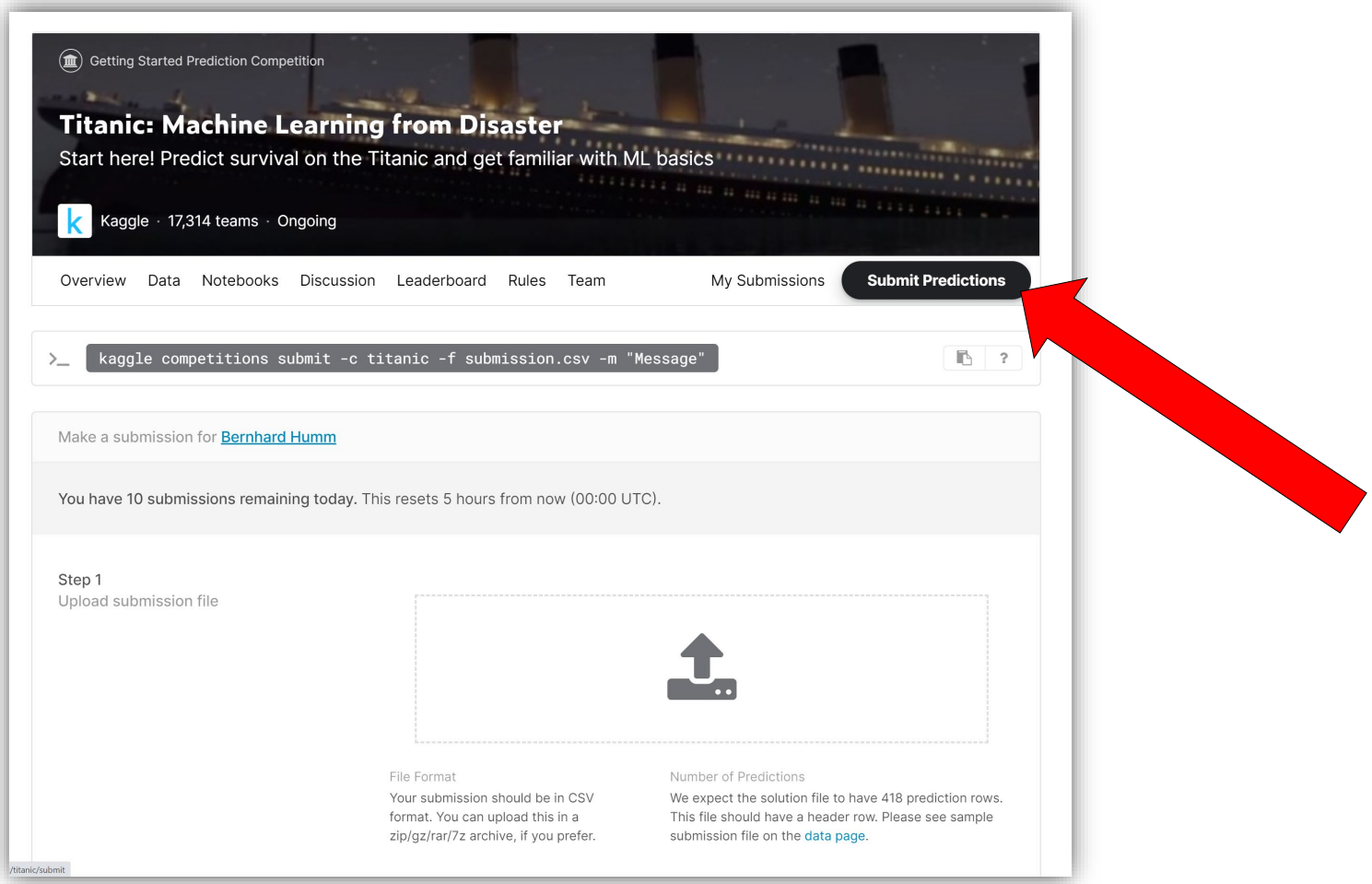
Generate Kaggle submission file

DataFrame with 2 columns:
PassengerID and Survived

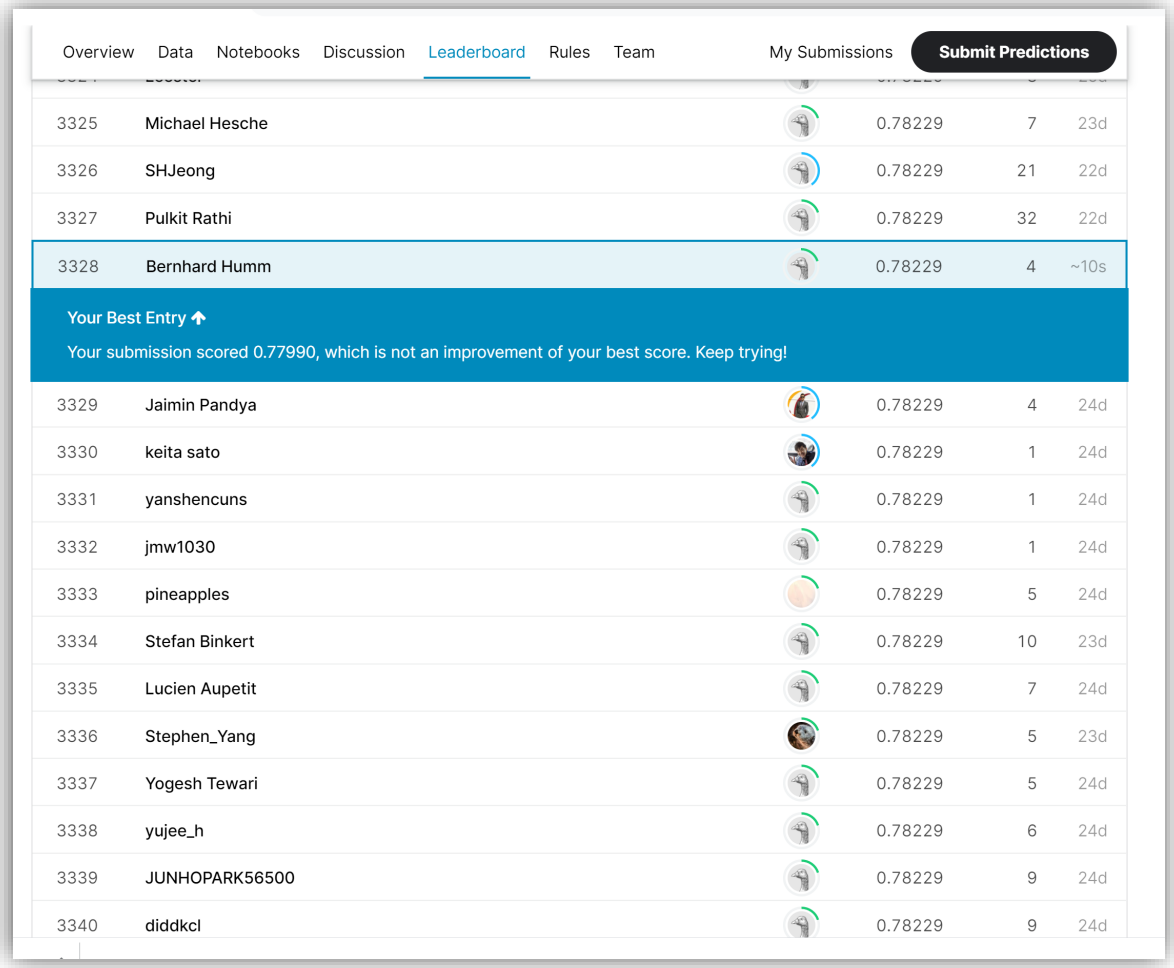
```
# prepare submission
output = pd.DataFrame({'PassengerId': test_data.PassengerId, 'Survived': predictions})
output.to_csv('data/submission.csv', index=False)
```

Write DataFrame to CSV file

Submit your solution to Kaggle



Check your position in the leaderboard



Overview Data Notebooks Discussion Leaderboard Rules Team My Submissions **Submit Predictions**

3325	Michael Hesche	0.78229	7	23d
3326	SHJeong	0.78229	21	22d
3327	Pulkit Rathi	0.78229	32	22d
3328	Bernhard Humm	0.78229	4	~10s

Your Best Entry ↑

Your submission scored 0.77990, which is not an improvement of your best score. Keep trying!

3329	Jaimin Pandya	0.78229	4	24d
3330	keita sato	0.78229	1	24d
3331	yanshencuns	0.78229	1	24d
3332	jmw1030	0.78229	1	24d
3333	pineapples	0.78229	5	24d
3334	Stefan Binkert	0.78229	10	23d
3335	Lucien Aupetit	0.78229	7	24d
3336	Stephen_Yang	0.78229	5	23d
3337	Yogesh Tewari	0.78229	5	24d
3338	yujee_h	0.78229	6	24d
3339	JUNHOPARK56500	0.78229	9	24d
3340	diddkcl	0.78229	9	24d

Agenda

- Overview
- ML applications
- ML tasks
- Example: Titanic survival classification
- Kaggle
- Analyzing dataset
- Loading data
- Preprocessing data
- ML training
- Predicting
- Quick check

Quick check
Machine Learning - Classification

1. What is ML?
2. When to use ML / when not to use ML?
3. Name applications of ML
4. Explain the ML area supervised learning
5. Explain the ML task classification
6. What is Kaggle?
7. Why is it important to analyse a dataset?
8. Explain categories and data types of dataset attributes
9. Why is pre-processing important? Give examples
10. What is ML training?
11. How to predict new cases?